Short Tutorial on R and Biostatistics

Biostatistics Material for: Evidence-Based Coaching of Core Competencies in Epidemiology using the Framework of Randomized Controlled Trials

Sarah Haile

2026-10-19

Table of contents

Preface 4
About this tutorial . . . 4

1 A note on setting up R and R studio 6

2 Statistical Methods for Continuous Outcomes 9
2.1 The Didgeridoo study . . . 9
2.2 Summarizing continuous outcomes . . . 10
2.3 Comparing continuous outcomes . . . 12

3 Clinically / practically relevant or statistical significant? 15

4 Statistical Methods for Binary Outcomes 19
4.1 the Birthweight data . . . 19
4.2 Summarizing binary outcomes . . . 22
4.2.1 risk difference . . . 22
4.2.2 risk ratio . . . 23
4.2.3 odds ratio . . . 23
4.2.4 Calculating the odds ratio . . . 24
4.3 Comparing binary outcomes . . . 25

5 Choice of statistical test 29

6 First steps in R 30

7 Data checking 33
7.0.1 Take a first look . . . 34
7.1 Check some basic numbers . . . 36
7.2 Q: How many participants are in the data? . . . 36
7.3 Q: How many of them have been randomized? . . . 36
7.4 Q: How many have the primary endpoint, the reverse digit span test? . . . 37
7.5 Q: How does age look? . . . 37

8 Data cleaning and Table 1 39
8.1 Make a demographics table . . . 41

9 Visualization using `ggplot2` 43

10 Comparing and testing outcomes 53

11 Back to the beginning: importing data 56

12 Summary 57

References 58

Preface

About this tutorial

This tutorial outlines some key statistical methods needed to analyze randomized controlled trials (RCTs), as conducted in our hands-on RCT courses (Radtke et al. (2024)), as well as tools in R to do these methods in practice:

- B102 ("Konzepte und Methoden der Epidemiologie") in the Master of Public Health program,
- BME361 ("Randomised trials – From lab experiments to large preventive trials") in the MSc program in Biomedicine, and
- EPI301 ("Introduction to Epidemiology") in the PhD programs in Epidemiology and Biostatistics; Clinical Science; and Care & Rehabilitation Science.

There are many potential study designs (e.g. two-arm parallel group, more than 2 parallel groups, crossover trials, cluster-randomized, case-control, …) and types of outcomes (e.g. continuous normally-distributed, continuous but not normal, binary, ordinal, categorical, survival). We focus on performing the following key steps in analyzing a medical study:

- summarizing binary and continuous outcomes
- comparing these outcomes in a two-sample setting.

Along the way, we'll cover p-values and confidence intervals. We do not aim to provide a comprehensive overview to statistical methodology, but to get you started with some methods that are applicable in many common situations. For more information on these topics, try these resources for statistical methods:

- *Essential Medical Statistics* Kirkwood and Sterne (2003)
- *Medizinische Statistik*: Held et al. (2013)
- the online script for the *Clinical Biostatistics* course
- *Statistical Issues in Drug Development*: Senn (2021)
- *Cross-over Trials In Clinical Research*: Senn (2002)

and R programming:

- *Hands-On Programming with R*: Grolemund (2014)
- *R for Data Science*: Wickham et al. (2023)

- *ggplot2: Elegant Graphics for Data Analysis*: Wickham (2021)

This tutorial is a work in progress! Please contact me at sarah.haile@uzh.ch if you have any comments or questions.

This work is licensed under CC BY-NC-ND 4.0

1 A note on setting up R and R studio

The sections on statistical methods and R programming can be done in either order. However, both use R code.

If you want to follow along and run the analysis in R yourself, you'll want to set up R and RStudio on your computer and install some key R packages.

1. **Install R**: Go to cran.r-project.org and download and install the R version for your operating system.
2. **Install RStudio**: Most people prefer to run R using RStudio. Head over to posit.co and download and install "RStudio Desktop" for your OS.
3. **Open Rstudio**
4. **Install packages**: In the console:
    - Install the `tidyverse` using `install.packages("tidyverse")`. These are several data cleaning packages, like `dplyr`, `tidyr`, and `forcats` that you'll need for many tasks. And it includes `ggplot2` for all your plots.
    - Install `gtsummary`, `install.packages("gtsummary")`, which helps you summarize your datasets, like for a Table 1, and regression models.

```
install.packages("tidyverse")
install.packages("gtsummary")
```

Usually when you load the `tidyverse` package, it gives you a message about which packages it has loaded. *This message has been suppressed throughout this tutorial.* It can also be helpful to set a `ggplot2` theme for all your plots using `theme_set()`. Here we use the black and white theme, `theme_bw()` with font size `base_size = 12`

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.2.1     v readr     2.2.0
v forcats   1.0.1     v stringr   1.6.0
v ggplot2   4.0.3     v tibble    3.3.1
v lubridate 1.9.5     v tidyr     1.3.2
v purrr     1.2.2
```

```
-- Conflicts --------------------------------------- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becor
```

```r
theme_set(theme_bw(base_size = 12))
library(gtsummary)
```

It would be a good idea to keep this code in a script. You can make a new one by clicking on the "+" button in the top left, or with `File > New File > R Script`. All code in this online resource can be easily copied one of two ways:

1. Using the `</>` tag at the top right of a section will give you all the R code for the section.
2. In any small section of code, there's a "clipboard" icon on the right side which you can use to copy that bit of text.

In other parts of this tutorial, we use the `patchwork`, `ggbeeswarm`, and `palmerpenguins` R packages.

```r
library(patchwork)
library(ggbeeswarm)
library(palmerpenguins)
```

We also use this space to show version details for R and all loaded packages:

```r
R.version.string
```

```
[1] "R version 4.6.1 (2026-06-24)"
```

```r
sessionInfo()
```

```
R version 4.6.1 (2026-06-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.6.2

Matrix products: default
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib;  LAPACK

locale:
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Zurich
tzcode source: internal

attached base packages:
[1] stats     graphics  grDevices utils     datasets  methods   base

other attached packages:
 [1] palmerpenguins_0.1.1 ggbeeswarm_0.7.3     patchwork_1.3.2
 [4] gtsummary_2.5.0      lubridate_1.9.5      forcats_1.0.1
 [7] stringr_1.6.0        dplyr_1.2.1          purrr_1.2.2
[10] readr_2.2.0          tidyr_1.3.2          tibble_3.3.1
[13] ggplot2_4.0.3        tidyverse_2.0.0

loaded via a namespace (and not attached):
 [1] gtable_0.3.6       jsonlite_2.0.0     compiler_4.6.1     tidyselect_1.2.1
 [5] scales_1.4.0       yaml_2.3.12        fastmap_1.2.0      R6_2.6.1
 [9] generics_0.1.4     knitr_1.51         pillar_1.11.1      RColorBrewer_1.1-
3
[13] tzdb_0.5.0         rlang_1.3.0        stringi_1.8.7      xfun_0.59
[17] S7_0.2.2           otel_0.2.0         timechange_0.4.0   cli_3.6.6
[21] withr_3.0.3        magrittr_2.0.5     digest_0.6.39      grid_4.6.1
[25] rstudioapi_0.19.0  hms_1.1.4          beeswarm_0.4.0     lifecycle_1.0.5
[29] vipor_0.4.7        vctrs_0.7.3        evaluate_1.0.5     glue_1.8.1
[33] farver_2.1.2       rmarkdown_2.31     tools_4.6.1        pkgconfig_2.0.3
[37] htmltools_0.5.9
```

2 Statistical Methods for Continuous Outcomes

2.1 The Didgeridoo study

To discuss statistical methods for continuous outcomes, we use data from the didgeridoo study (Puhan et al. (2006)). In this RCT, participants with obstructive sleep apnea were randomized either to didgeridoo lessons with daily practice or to a waiting list for such lessons (control arm).

The primary endpoint is the Epworth Sleepiness Scale (ESS), a measure of daytime sleepiness on a scale from 0 (not sleepy in any situation) to 24 (very likely to fall asleep in all situations). We have data on this outcome at baseline (`epworth1`) and at 4 months (`epworth2`). (The data have been simplified for this tutorial.)

```
library(tidyverse)
theme_set(theme_bw(base_size = 12))
library(gtsummary)

# didgeridoo <- read_csv("data/alphorn.csv") |>
#   mutate(trt = factor(group, 1:0, c("control", "didgeridoo"))) |>
#   select(id, trt, age, gender, bmi, epworth1, epworth2)
```

```
didgeridoo <- tibble(id = 1:25,
                     trt = c(1, 2, 1, 2, 1, 2, 1, 1, 2, 2, 2, 2, 1, 1, 1,
                             2, 2, 2, 1, 1, 2, 2, 1, 2, 2),
                     age = c(45, 41, 41, 39, 33, 55, 50, 65, 43, 59, 53,
                             54, 49, 43, 56, 41, 53, 46, 39, 43, 50, 57, 53, 57, 51),
                     gender = c(0, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0,
                                0, 0, 0, 0, 0, 0, 0, 0, 0, 0),
                     bmi = c(26.1, 19, 27.2, 22.5, 29.4, 29.4, 26.8, 26.4, 31,
                             23.4, 29.4, 28.4, 26.6, 26.7, 19.4, 25.8, 25.7, 27.3,
                             24.5, 26, 27.2, 30, 25.9, 24.5, 18.2),
                     epworth1 = c(11, 11, 5, 8, 1, 9, 10, 11, 7, 9, 10, 12, 11,
                                  6, 10, 12, 11, 19, 13, 23, 18, 12, 21, 12, 15),
```

```
                      epworth2 = c(10, 5, 4, 8, 1, 7, 14, 6, 6, 9, 3, 11, 10, 5,
                                   8, 3, 9, 9, 9, 18, 8, 9, 21, 8, 8)) |>
  mutate(trt = factor(trt, 1:2, c("control", "didgeridoo")))

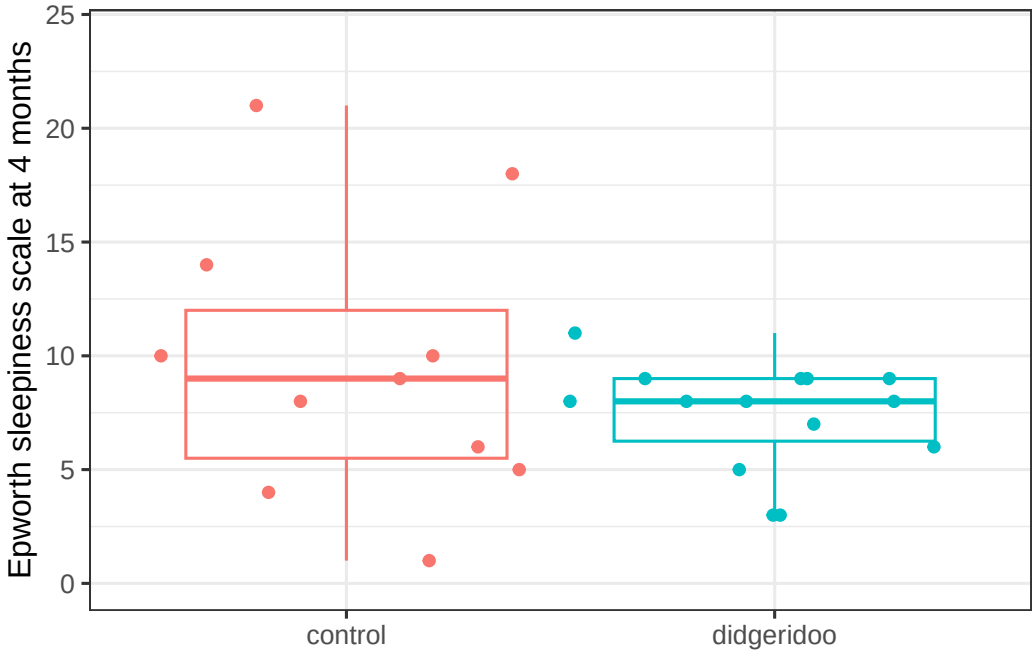
didgeridoo
```

```
# A tibble: 25 x 7
      id trt          age gender   bmi epworth1 epworth2
   <int> <fct>      <dbl>  <dbl> <dbl>    <dbl>    <dbl>
 1     1 control       45      0  26.1       11       10
 2     2 didgeridoo    41      1  19         11        5
 3     3 control       41      0  27.2        5        4
 4     4 didgeridoo    39      0  22.5        8        8
 5     5 control       33      0  29.4        1        1
 6     6 didgeridoo    55      1  29.4        9        7
 7     7 control       50      1  26.8       10       14
 8     8 control       65      1  26.4       11        6
 9     9 didgeridoo    43      0  31          7        6
10    10 didgeridoo    59      0  23.4        9        9
# i 15 more rows
```

2.2 Summarizing continuous outcomes

A graph of ESS at 4 months, `epworth2`, by treatment could look something like this:

```
ggplot(aes(trt, epworth2,
           color = trt), data = didgeridoo) +
  geom_jitter(height = 0, width = 0.5) +
  geom_boxplot(fill = NA, outliers = FALSE) +
  guides(color = "none") +
  ylim(0, 24) +
  xlab(NULL) + ylab("Epworth sleepiness scale at 4 months")
```



It looks like the didgeridoo group has somewhat lower ESS, on average, than the control group. The next step is to quantify how much lower. A quick way is just to use `group_by()` and `summarize()`.

```
didgeridoo |>
  group_by(trt) |>
  summarize(m = mean(epworth2))
```

```
# A tibble: 2 x 2
  trt            m
  <fct>      <dbl>
1 control     9.64
2 didgeridoo  7.36
```

Using `tbl_summary()` extends nicely to more than 1 endpoint:

```
didgeridoo |>
  select(trt, epworth2) |>
  tbl_summary(by = trt,
              statistic = list("epworth2" = "{mean} ({sd})"))
```

Either way, we get the same results: average / mean ESS in the didgeridoo group is 7.4 compared to 9.6 in the control group. The difference between the two groups is therefore -2.2,

Characteristic	**control** N = 11[1]	**didgeridoo** N = 14[1]
epworth2	9.6 (6.0)	7.4 (2.3)

[1]Mean (SD)

that is, participants taking didgeridoo lessons had on average a 2.2 point lower ESS than those not taking lessons.

```
7.4 - 9.6
```

```
[1] -2.2
```

2.3 Comparing continuous outcomes

Now we want to test for difference in ESS between the two arms in the didgeridoo study. One option for that is a two-sample t-test.

```
t.test(epworth2 ~ trt, data = didgeridoo)
```

```
    Welch Two Sample t-test

data:  epworth2 by trt
t = 1.187, df = 12.381, p-value = 0.2575
alternative hypothesis: true difference in means between group control and group didgeridoo i
95 percent confidence interval:
 -1.890289  6.448731
sample estimates:
   mean in group control mean in group didgeridoo
                9.636364                 7.357143
```

(Note: You may have learned in a statistics class that there are several versions of the t-test, depending on whether the variances and / or sample sizes are similar. As it is not recommended to test for equal variances first (Zimmerman (2004)), `t.test()` conducts the most general version, "Welch", of the t-test by default (i.e., without the equal variance assumption, using the option `var.equal = FALSE`).)

Previously, we had a treatment difference of `7.4 - 9.6 = -2.2`. Now we also get a 95% confidence interval from -1.89 to 6.45, and p = 0.26. Both indicate no difference between intervention and control.

Characteristic	Beta	95% CI	p-value
trt			
control	—	—	
didgeridoo	-2.3	-5.9, 1.3	0.2

Abbreviation: CI = Confidence Interval

But wait, shouldn't the treatment effect be inside the confidence interval. Do you see why that is not here?

In R, as in Stata and other statistical programming languages, the `t.test` results compare the 2nd group to the 1st group, or in this case, `control - didgeridoo`. This is the opposite of what we calculated above `didgeridoo - control`, and makes the `t.test` output somewhat difficult to follow.

```
# mean difference from t.test()
9.636 - 7.357
```

```
[1] 2.279
```

We could also use linear regression, `lm()`, to get the same results, and then `tbl_regression()` to show the coefficient table with confidence intervals.

```
mod <- lm(epworth2 ~ trt, data = didgeridoo)
tbl_regression(mod, conf.int = TRUE)
```

The results from linear regression may not be exactly the same as those from the t-test, but they are quite similar. Additionally, linear regression allows us to further adjust for other participant characteristics, e.g. baseline ESS.

```
mod <- lm(epworth2 ~ trt + epworth1, data = didgeridoo)
tbl_regression(mod, conf.int = TRUE)
```

After adjustment for baseline ESS, the observed treatment effect is -2.7 (95% CI -5.1 to -0.35, p = 0.027), showing a modest and statistically significant effect of didgeridoo playing on sleep apnea. These are essentially the results that Puhan et al. (2006) reported, though they examined change scores.

Characteristic	Beta	95% CI	p-value
trt			
control	—	—	
didgeridoo	-2.7	-5.1, -0.35	0.027
epworth1	0.67	0.42, 0.92	<0.001

Abbreviation: CI = Confidence Interval

Characteristic	Beta	95% CI	p-value
trt			
control	—	—	
didgeridoo	-3.0	-5.7, -0.27	0.033

Abbreviation: CI = Confidence Interval

```
lm((epworth2 - epworth1) ~ trt, data = didgeridoo) |>
  tbl_regression(conf.int = TRUE)
```

3 Clinically / practically relevant or statistical significant?

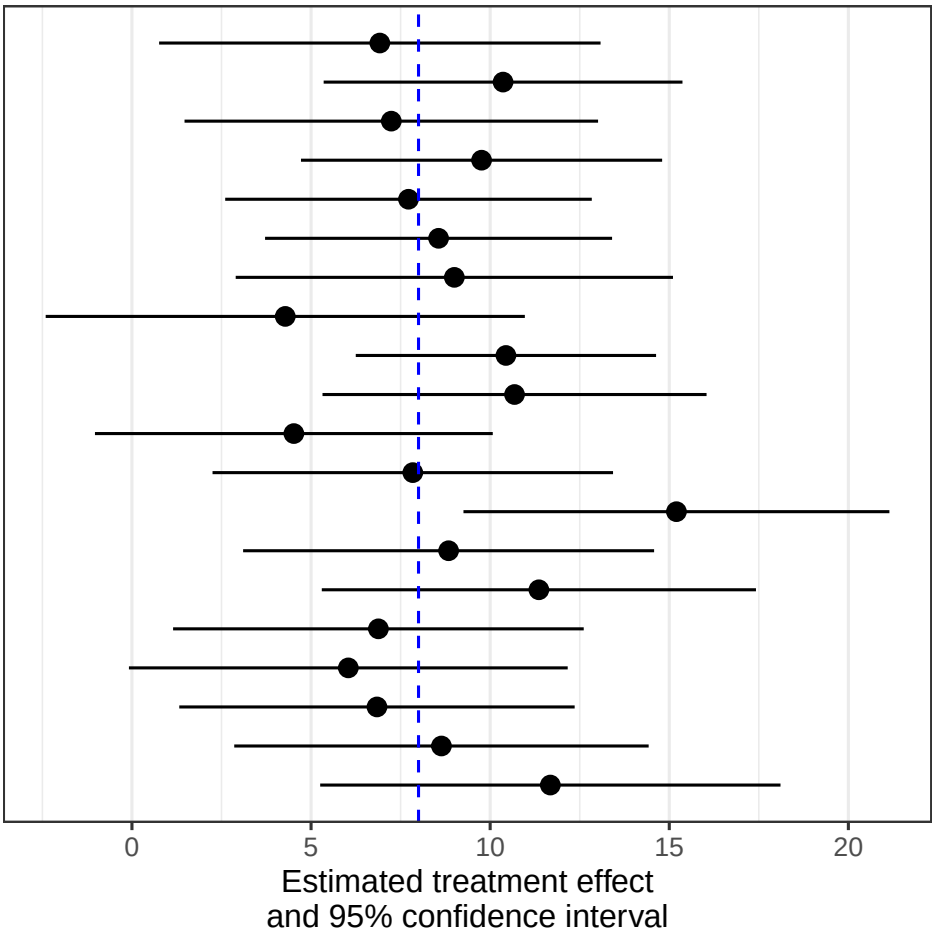
In our analysis of the didgeridoo study, we compared ESS as the average difference between the two groups. Such a comparison is often called an **effect size** or **treatment effect**. It quantifies the observed association between the treatment and outcome, and as such, gives us information about the clinical or practical relevance of the intervention of didgeridoo lessons on the outcome ESS.

Now, if we conducted the didgeridoo study again, even in the same study center and with the same didgeridoo instructor, we would not get exactly the same result, would we? Perhaps it would be close, but it's unlikely to be exactly the same. We might like to guess what other possible treatment effects would be plausible. For that, we need a **confidence interval** (CI), which describes a range of most plausible treatment effects given our observed data, describing clinical or practical relevance. If a treatment effect corresponding to no difference (e.g. mean difference of 0) is included in the range of the CI, we generally say that the study results are consistent with no difference between the treatment arms.

You may have seen the phrase "95% confidence interval" before. Why can't we just say "confidence interval"? The "95%" part quantifies *how* confident we want to be that our confidence interval contains the true population-level treatment effect. However, it's not very often you will see anything except a 95% confidence interval.

Suppose we conduct the same trial 20 times for an intervention that has a true effect of 8 points on the outcome of interest. The plot below shows the estimated treatment effects for each of those studies, as well as the corresponding 95% confidence intervals. From these 20 studies, almost all confidence intervals, that is 19/20 or 95%, contain the true population-level treatment effect, so we can be reasonably confident that the CI we observed from our single trial includes the true effect we would like to estimate.

Suppose we conduct the same trial 20 times for an intervention that has a true effect of 8 points on the outcome of interest. The plot below shows the estimated treatment effects for each of those studies, as well as the corresponding 95% confidence intervals. From these 20 studies, almost all confidence intervals, that is 19/20 or 95%, contain the true population-level treatment effect, so we can be reasonably confident that the CI we observed from our single trial includes the true effect we would like to estimate.



And finally, we have **p-values**, a measure of *statistical significance*. Now imagine we have an intervention which does not affect the endpoint compared to the control arm.

- If the difference is small, and we think that most studies would give us results at least this size or larger (assuming no difference between intervention and control), we say that the results are "not suprising", and conclude no difference.
- If the difference is large, and we think that few studies would give us results at least this size or larger (assuming no difference between intervention and control), we say that the results are "suprising", and conclude there is a difference between the treatment arms.

P-values quantify how surprised we would be by the results, if we assume that there is no difference between intervention and control. Imagine for a moment that a friend drops by with a 2 sets of 10 dice, giving you the 10 blue dice, and keeping the 10 red dice for himself. "The game is easy. Whoever gets the most 6s out of the 10 dice wins." You roll your blue dice and get exactly one 6s. He rolls his red dice and gets 5 6s! That's amazing! Later you think about it again: if his red dice were fair dice, 5 6s is a really surprising outcome. Was

he cheating? How surprising is it exactly, to get 5 6s from a set of 10 fair dice? After a bit of calculation, you discover that getting 5 or more 6s from 10 dice only should happen about 1.5% of the time, assuming you have fair dice. You're not sure, but it looks like he had some sort of special dice, or was very very lucky. Maybe you'll have to get him to try the game again another day to find out which it is.

Back to clinical trials: In regulatory settings, we usually say our level of surprise should be no more than 5% ("significance level"):

- If $p < 0.05$, that is, if the observed results would be expected no more than 5% of the time when there is actually no difference between intervention and control, then we say the results are *statistically significant".
- If $p \geq 0.05$, that is, if the observed results would be expected more than 5% of the time when there is actually no difference between intervention and control, then we say the results are *not statistically significant".

Power and sample size considerations are generally based on this simple decision rule. In other situations, we do not need clear predefined rules for when the study results are considered "positive". Then it may be more helpful to consider p-values as measures of evidence.

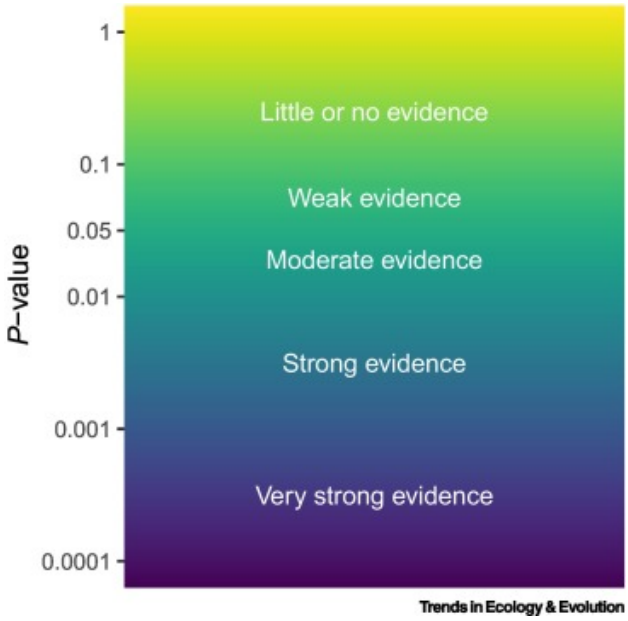


Figure 3.1: Instead of reporting a binary yes/no test outcome, the results sections of scientific papers should rather report the exact P-values and interpret that 'there was no/weak/moderate/strong/very strong evidence' for a certain finding or effect, depending on approximate ranges into which the actual P-value falls (Muff et al 2022)

Regardless of how we interpret p-values, they are measures of statistical significance only, and make no statement about the clinical relevance of the observed results. In very small studies, p-values will be mostly not significant, even if the results are practically meaningful. On the other hand, in very large studies, p-values will generally be significant, even if the observed effects are very small.

For an overview of p-values and confidence intervals, find a good statistical methods textbook (e.g., Kirkwood and Sterne (2003) or Held et al. (2013)), or take a look at papers like Rovettaa et al. (2025), Greenland et al. (2016) or Muff et al. (2022).

4 Statistical Methods for Binary Outcomes

Now that we've looked at some typical methods for continuous outcomes, we switch to *binary* outcomes. These outcome generally answer some yes/no kind of question:

- Did the participant finish the test?
- Did the patient have a heart attack?

4.1 the Birthweight data

In birthweight data, available from the `MASS` package, we want to examine whether babies of mothers who smoke were more likely to be of low birth weight, than of mothers who did not smoke.

```
library(tidyverse)
theme_set(theme_bw(base_size = 12))
library(gtsummary)

birthweight <- as_tibble(MASS::birthwt) |>
  mutate(race = factor(race, 1:3, c("white", "black", "other")),
         smoke = factor(smoke, 0:1, c("non-smoker", "smoker")))

birthweight
```

```
# A tibble: 189 x 10
     low   age   lwt race  smoke        ptl    ht    ui   ftv   bwt
   <int> <int> <int> <fct> <fct>      <int> <int> <int> <int> <int>
 1     0    19   182 black non-smoker     0     0     1     0  2523
 2     0    33   155 other non-smoker     0     0     0     3  2551
 3     0    20   105 white smoker         0     0     0     1  2557
 4     0    21   108 white smoker         0     0     1     2  2594
 5     0    18   107 white smoker         0     0     1     0  2600
 6     0    21   124 other non-smoker     0     0     0     0  2622
 7     0    22   118 white non-smoker     0     0     0     1  2637
 8     0    17   103 other non-smoker     0     0     0     1  2637
```

Characteristic	**non-smoker** N = 115[1]	**smoker** N = 74[1]
low	29.0 (25.2%)	30.0 (40.5%)

[1] n (%)

```
 9     0    29   123 white smoker         0     0     0     1  2663
10     0    26   113 white smoker         0     0     0     0  2665
# i 179 more rows
```

First, we summarize the proportion of low birthweight babies by mothers' smoking status.

```
birthweight |>
  select(low, smoke) |>
  tbl_summary(by = smoke,
              digits = list("low" ~ 1))
```

Among the 74 smoking mothers, 41% (n = 30) babies were of low birthweight, compared to 25% (n = 115) from 115 non-smoking mothers.

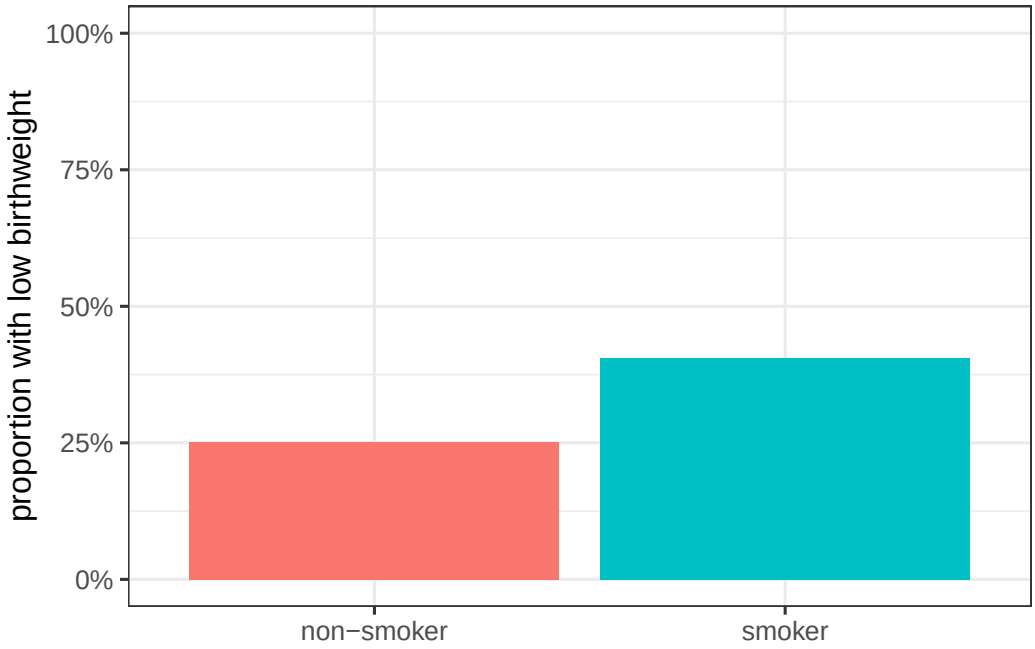
A simple graph for binary outcomes might look like this.

```
tmp <- birthweight |>
  group_by(smoke) |>
  summarize(n = n(),
            lowbw = sum(low),
            pct = lowbw / n)

tmp
```

```
# A tibble: 2 x 4
  smoke          n lowbw   pct
  <fct>      <int> <int> <dbl>
1 non-smoker   115    29 0.252
2 smoker        74    30 0.405
```

```
ggplot(aes(smoke, pct, fill = smoke), data = tmp) +
  geom_col() +
  scale_y_continuous(limits = 0:1,
                     labels = scales::percent) +
  guides(fill = "none") +
  xlab(NULL) + ylab("proportion with low birthweight")
```



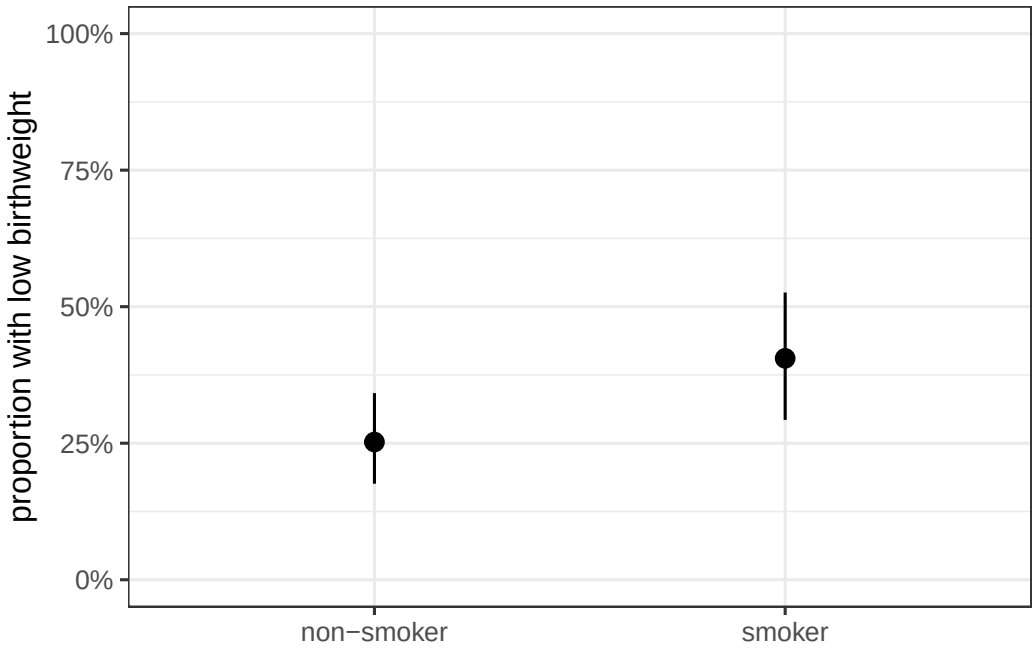
As an alternative that also accounts for variation and sample size, this version with confidence intervals is also nice.

```
library(purrr)
tmp <- birthweight |>
  group_by(smoke) |>
  summarize(n = n(),
            lowbw = sum(low),
            pct = lowbw / n) |>
  mutate(bt = map2(lowbw, n, ~ binom.test(.x, .y)),
         lb = map_dbl(bt, ~ .$conf.int[1]),
         ub = map_dbl(bt, ~ .$conf.int[2]))

tmp
```

```
# A tibble: 2 x 7
  smoke          n lowbw   pct bt         lb    ub
  <fct>      <int> <int> <dbl> <list>  <dbl> <dbl>
1 non-smoker   115    29 0.252 <htest> 0.176 0.342
2 smoker        74    30 0.405 <htest> 0.293 0.526
```

```
ggplot(aes(smoke, pct,
           ymin = lb, ymax = ub), data = tmp) +
  geom_pointrange() +
  scale_y_continuous(limits = 0:1,
                     labels = scales::percent) +
  xlab(NULL) + ylab("proportion with low birthweight")
```



4.2 Summarizing binary outcomes

For binary outcomes, we have a few options.

4.2.1 risk difference

The **risk difference** is calculated as `outcome proportion in the intervention group` minus `outcome proportion in the control group`.

```
0.405 - 0.252
```

```
[1] 0.153
```

risk difference: Smoking mothers are 15% more likely to have a low birthweight baby than non-smoking mothers.

4.2.2 risk ratio

The **risk ratio** is calculated as `outcome proportion in the intervention group` divided by `outcome proportion in the control group`.

```
0.405 / 0.252
```

```
[1] 1.607143
```

risk ratio: Smoking mothers are 1.6 times more likely to have a low birthweight baby than non-smoking mothers.

4.2.3 odds ratio

Finally, the **odds ratio** is calculated as `outcome odds in the intervention group` divided by `outcome odds in the control group`. But what is *odds*? Odds compare the probability that an event will happen to the probability that it will not happen. If the probability of an event is 75%, we say the odds of that event are 75%/25%, equal to "3 to 1" or just 3. For some other probabilities we get:

Table 4.1: probabilities and odds

p	1 - p	odds	odds (alternative)
0.80	0.20	4.000	4 to 1
0.75	0.25	3.000	3 to 1
0.70	0.30	2.333	7 to 3
0.60	0.40	1.500	3 to 2
0.50	0.50	1.000	1 to 1
0.40	0.60	0.667	2 to 3
0.30	0.70	0.429	3 to 7
0.25	0.75	0.333	1 to 3
0.20	0.80	0.250	1 to 4

A probability of 40% (as we have for low birthweight babies of smoking mothers) indicates odds of 0.667. Similarly, a probability of 25% (for low birthweight babies of non-smoking mothers) corresponds to odds of 0.333. The odds ratio is therefore about 2.

	smoke		
	non-smoker	smoker	Total
low			
0	86.0 (74.8%)	44.0 (59.5%)	130.0 (68.8%)
1	29.0 (25.2%)	30.0 (40.5%)	59.0 (31.2%)
Total	115.0 (100.0%)	74.0 (100.0%)	189.0 (100.0%)

```
(0.4 / 0.6) / (0.25 / 0.75)
```

```
[1] 2
```

odds ratio: Smoking mothers gave 2 times the odds to have a low birthweight baby than non-smoking mothers.

4.2.4 Calculating the odds ratio

You may have noticed we weren't very precise about calculating the odds ratio just now. In this section, we'll show some basic calculations to get the correct odds ratio. The simplest way to summarize these kind of yes/no outcomes by group is the so-called 2x2 table (Bland and Altman (2000)).

```
birthweight |>
  tbl_cross(col = smoke,  row = low,
            percent = "column", digits = 1)
```

For non-smokers, we see that the `29/115 = 0.252` of non-smoking mothers had low birthweight babies, compared to `30/74 = 0.405` of smoking mothers. To get odds, we calculate

```
29 / 86
```

```
[1] 0.3372093
```

for non-smoking mothers, and

```
30 / 44
```

```
[1] 0.6818182
```

for smoking mothers. We do not need the column totals to calculate odds.

To get the odds ratio, we then calculate:

```
(30 / 44) / (29 / 86)
```

```
[1] 2.021944
```

That's a lot of division signs! But perhaps you remember from school that instead of dividing by the 2nd fraction, we can multiply by its reciprocal.

$$\frac{30}{44} \Big/ \frac{29}{86} = \frac{30}{44} \times \frac{86}{29} = \frac{86 \cdot 30}{29 \cdot 44} = 2.02$$

To remember this way of getting the odds ratio, you multiply the numbers on the diagonal, `86 * 30`, and then divide by those not on the diagonal, `44 * 29`.

4.3 Comparing binary outcomes

To compare these rates with a hypothesis test, we could use a chi-square test, `chisq.test()`. In order to run this test in R, we first need to set up the 2x2 table. Additionally, since `chisq.test()` only reports a p-value, we need to find another way to get the odds ratio, which we can do with the `epitools` package.

```
library(epitools)
tab <- with(birthweight, table(low, smoke))
tab
```

```
   smoke
low non-smoker smoker
  0         86     44
  1         29     30
```

```
chisq.test(tab)
```

```

    Pearson's Chi-squared test with Yates' continuity correction

data:  tab
X-squared = 4.2359, df = 1, p-value = 0.03958
```

```r
oddsratio(tab)
```

```
$data
       smoke
low     non-smoker smoker Total
  0             86     44   130
  1             29     30    59
  Total        115     74   189

$measure
   odds ratio with 95% C.I.
low estimate    lower    upper
  0  1.00000       NA       NA
  1  2.01268 1.073703 3.794579

$p.value
   two-sided
low midp.exact fisher.exact chi.square
  0         NA           NA         NA
  1 0.02914865    0.0361765 0.02649064

$correction
[1] FALSE

attr(,"method")
[1] "median-unbiased estimate & mid-p exact CI"
```

There are small differences between the output of `chisq.test()` and `oddsratio()` here, but that's because one uses a "continuity correction" by default and the other does not.

```r
oddsratio(tab, correction = TRUE)
```

```
$data
       smoke
low     non-smoker smoker Total
  0             86     44   130
  1             29     30    59
  Total        115     74   189

$measure
   odds ratio with 95% C.I.
```

Characteristic	OR	95% CI	p-value
smoke			
non-smoker	—	—	
smoker	2.02	1.08, 3.80	0.028

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

```
low estimate    lower    upper
  0  1.00000       NA       NA
  1  2.01268 1.073703 3.794579

$p.value
   two-sided
low midp.exact fisher.exact chi.square
  0         NA           NA         NA
  1 0.02914865    0.0361765 0.03957697

$correction
[1] TRUE

attr(,"method")
[1] "median-unbiased estimate & mid-p exact CI"
```

This approach is admittedly a bit tedious. We could also use logistic regression, which can be done with the `glm()` function, and the option `family = binomial`. Also, when we look at the regression coefficients, we need to exponentiate them, $e^{\widehat{\beta}}$, to get odds ratios.

```
mod <- glm(low ~ smoke, data = birthweight,
           family = binomial)
tbl_regression(mod, conf.int = TRUE, exponentiate = TRUE)
```

Logistic regression also allows us to adjust for other factors, like mother's age or race.

```
mod <- glm(low ~ smoke + age + race, data = birthweight, family = binomial)
tbl_regression(mod, conf.int = TRUE, exponentiate = TRUE)
```

Characteristic	**OR**	**95% CI**	**p-value**
smoke			
non-smoker	—	—	
smoker	3.01	1.47, 6.38	0.003
age	0.97	0.90, 1.03	0.3
race			
white	—	—	
black	2.75	1.04, 7.31	0.040
other	2.88	1.32, 6.53	0.009

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

5 Choice of statistical test

If you've been in a statistical methods course, you may remember that there were many methods presented. How do we know which one to pick?

For a relatively simple randomized clinical trial with

- 2 treatment arms
- Participants are independent
- Primary endpoint at 1 timepoint, either (approximately) continuous or binary

we have the following options:

type of outcome	**effect estimate**	**not adjusted**	**adjusted**
continuous *	mean difference	t-test	linear regression
binary (yes / no)	odds ratio	χ-square test	logistic regression
…	…	…	…

Note: *Continuous outcomes usually need to be only approximately normally distributed.*

Selection of statistical methods is however usually more complicated than this! A competent statistician can help you determine the most appropriate methods for your study.

6 First steps in R

Even without any data loaded, we can use R like a calculator. If we type `2 + 2` and "Run Code" (in R Studio: `Ctrl / CMD + Enter`), it calculates the answer:

```
2 + 2
```

```
[1] 4
```

> Where do I type this?
>
> Look for the "Console" in RStudio. By default it's in the pane on the bottom left. You can move it, by going to `View > Panes > Console on Right`.

Some other mathematical operators you probably recognize are:

- `-` (for subtraction or negative numbers),
- `*` (multiplication),
- `/` (division), and
- `^` (power, as in 3^2 = `3^2`).

There are also functions for

- square root (`sqrt()`),
- rounding to say 1 digit (`round(..., digits = 1)`),
- logarithm (`log()`, or base 10, `log10()`),
- exponential (`exp()`).

Combining these operators, along with some parentheses, allows us to calculate many things. For example, calculate BMI (body mass index) for a person who is 180cm tall and weighs 85 kg.

```
85 / (1.8^2)
```

```
[1] 26.23457
```

We can also save these height and weight values with an arrow, `<-`, and use them later for the calculation.

```
wt <- 85
ht <- 180
wt / ((ht / 100) ^2)
```

```
[1] 26.23457
```

But what if we had height and weight for, say 3 people?

- Person 1: weight 85, height 180
- Person 2: weight 90, height 190
- Person 3: weight 95, height 200

It would be annoying, not to mention error-prone, to have to calculate each BMI separately. To save all 3 heights or weights at once, we use a vector, by putting all 3 values inside `c()`, separated by commas: `c(85, 90, 95)`.

```
wt <- c(85, 91, 95)
ht <- c(180, 190, 200)
wt / ((ht / 100)^2)
```

```
[1] 26.23457 25.20776 23.75000
```

Now that we're working with more than one observation, we might also need functions like:

- mean (`mean()`),
- standard deviation (`sd()`),
- median (`median()`),
- sum (`sum()`), or
- minimum and maximum (`min()`, `max()`).

```
wt <- c(85, 90, 95)
ht <- c(180, 190, 200)
bmi <- wt / ((ht / 100)^2)
mean(bmi)
```

```
[1] 24.97177
```

Combining this information in a dataset, and then performing the calculation is more efficient. We'll talk more about that in the next two sections.

```
library(tidyverse)
```

```
mydata <- tibble(id = c(1, 2, 3),
                 wt = c(85, 90, 95),
                 ht = c(180, 190, 200)) |>
  mutate(bmi = wt / ((ht / 100)^2))
mydata
```

```
# A tibble: 3 x 4
     id    wt    ht   bmi
  <dbl> <dbl> <dbl> <dbl>
1     1    85   180  26.2
2     2    90   190  24.9
3     3    95   200  23.8
```

7 Data checking

```
library(tidyverse)
```

Now, imagine we have just completed a randomized clinical trial.

- **Population**: students in this course
- **Intervention**: watching a concentration video for 5 minutes
- **Control**: watching a screensaver-type video for 5 minutes
- **Outcome**: reverse digit span test

The primary outcome is the reverse digit span test, a computer-based test of memory (high number of digits is better). Secondary outcomes are the cold pressor test (number of seconds you can keep your hand in ice cold water), and a video challenge, that participants either complete or do not.

> **Note**
>
> This dataset is not real, just simulated. If you want to use it, unfold the code here.

```
set.seed(20261019)
n <- 60
dat <- tibble(study_id = 1:(2 * n),
              first = sample(LETTERS, 2 * n, replace = TRUE),
              last = sample(LETTERS, 2 * n, replace = TRUE),
              inclusion_over17 = 1,
              inclusion2 = 1, inclusion3 = 1, inclusion4 = 1,
              exclusion1 = 0, exclusion2 = 0,
              screened = 1,
              informed_consent = rbinom(2 * n, 1, 0.9),
              endpoints =  rbinom(2 * n, 1, 0.9),
              age_today = floor(runif(2 * n, 25, 45)),
              gender = rbinom(2 * n, 2, 0.250),
              video_games = rbinom(2 * n, 4, 0.2),
              randomization_group = rep(0:1, n)) |>
  mutate(reverse_digit = rnorm(2 * n, 3 + 2 * randomization_group, 1.5),
```

```
         reverse_digit = round(reverse_digit),
         cold_pressor = rnorm(2 * n, 60 - 10 * randomization_group, 20),
         cold_pressor = round(cold_pressor),
         video_challenge = rbinom(2 * n, 1, 0.2 + 0.3 * randomization_group)) |>
  mutate(across(age_today:video_challenge, ~ ifelse(informed_consent == 1, .x, NA))) |>
  mutate(across(reverse_digit:video_challenge, ~ ifelse(endpoints == 1, .x, NA))) |>
  select(-endpoints)

sm <- dat |>
  select(-(inclusion_over17:screened)) |>
  rename(id = study_id,
         trt = randomization_group,
         age = age_today) |>
  mutate(gender = factor(gender, 0:2, c("F", "M", "O")),
         trt = factor(trt, 0:1, c("control", "intervention")),
         video_games = factor(video_games, 0:3,
                              c("<30 minutes per day", "30-60", "60-120", "120+"))) |>
  filter(!is.na(trt), !is.na(reverse_digit))
```

7.0.1 Take a first look

We have exported the study data from redcap, imported it into R, and named it `dat`. Use the functions `head()` or `glimpse()` to look at the data structure (or `View()` if you're coding along in RStudio), `names()` to see what variables there are.

```
glimpse(dat)
```

```
Rows: 120
Columns: 18
$ study_id            <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,~
$ first               <chr> "F", "T", "L", "G", "N", "O", "M", "L", "B", "Y", ~
$ last                <chr> "O", "G", "Y", "L", "E", "J", "L", "H", "O", "X", ~
$ inclusion_over17    <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ inclusion2          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ inclusion3          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ inclusion4          <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ exclusion1          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ exclusion2          <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,~
$ screened            <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,~
$ informed_consent    <int> 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1,~
$ age_today           <dbl> 34, 43, NA, 26, 26, 42, 27, 44, 44, 32, NA, 31, 43~
```

```
$ gender              <int> 0, 0, NA, 0, 0, 0, 0, 1, 1, 1, NA, 1, 0, 0, 1, 1, ~
$ video_games         <int> 0, 0, NA, 0, 0, 2, 2, 2, 1, 1, NA, 0, 1, 1, 0, 2, ~
$ randomization_group <int> 0, 1, NA, 1, 0, 1, 0, 1, 0, 1, NA, 1, 0, 1, 0, 1, ~
$ reverse_digit       <dbl> 3, 2, NA, 3, 3, 4, 5, 6, 4, 5, NA, 5, 4, 4, 3, 6, ~
$ cold_pressor        <dbl> 75, 31, NA, 47, 46, 43, 78, 52, 54, 83, NA, 35, 76~
$ video_challenge     <int> 0, 1, NA, 1, 0, 1, 1, 1, 0, 1, NA, 0, 0, 1, 0, 1, ~
```

```
# View(dat)
names(dat)
```

```
 [1] "study_id"            "first"               "last"
 [4] "inclusion_over17"    "inclusion2"          "inclusion3"
 [7] "inclusion4"          "exclusion1"          "exclusion2"
[10] "screened"            "informed_consent"    "age_today"
[13] "gender"              "video_games"         "randomization_group"
[16] "reverse_digit"       "cold_pressor"        "video_challenge"
```

Here is a simplified version of what you might find in the REDCap codebook for this dataset:

variable name	description	coding
`study_id`	participant number	
`first` / `last`	first and last initials	
`inclusion...`	4 variables with different inclusion criteria	should all be 1
`exclusion...`	2 variables with different exclusion criteria	should all be 0
`screened`	screening complete?	1 = yes, 0 = no
`informed_consent`	participant signed informed consent?	1 = yes, 0 = no
`age_today`	State your age today.	
`gender`	State your gender.	0 = female, 1 = male, 2 = other
`video_games`	How long do you play video games on an average day?	0 = <30 minutes, 1 = 30-60 minutes, 2 = 60-120 minutes, 3 = more than 120 minutes.
`randomization_group`	Randomized intervention	0 = control, 1 = intervention
`reverse_digit`	reverse digit span test	number of digits remembered

variable name	description	coding
`cold_pressor`	cold pressor test	number of seconds participant kept their hand in the ice water
`video_challenge`	could the participant finish the short video challenge?	1 = yes, 0 = no

7.1 Check some basic numbers

Next, we run some basic checks to see how many participants have been included, how many have been randomized, and how many have complete the primary endpoint.

7.2 Q: How many participants are in the data?

Use `nrow()` to find out how many participants have been screened.

```
nrow(dat)
```

```
[1] 120
```

7.3 Q: How many of them have been randomized?

A common thing to check when you're first looking at a dataset is how many of something there are. How many participants? How many participanted have been randomized? How many are at least 18 years old? For these tasks, we need `count()`. Using `dplyr` coding style, we write the name of the dataset, `dat`, write "then" using the so-called "pipe" `|>` (or `%>%`, either way it's ctrl / cmd + M), and finally `count()`.

```
dat |>
  count()
```

```
# A tibble: 1 x 1
      n
  <int>
1   120
```

This count should be the same as we got above with `nrow()`.

We want to see how many participants are in each category of `randomization_group`, or where it is missing. So let's `count()` `randomization_group`.

```
dat |>
  count(randomization_group)
```

```
# A tibble: 3 x 2
  randomization_group     n
                <int> <int>
1                   0    54
2                   1    58
3                  NA     8
```

7.4 Q: How many have the primary endpoint, the reverse digit span test?

The `is.na()` function checks if a value is missing (`NA`). To check if it's not missing, use `!is.na()` (`!` means "not").

```
dat |>
  count(!is.na(reverse_digit))
```

```
# A tibble: 2 x 2
  `!is.na(reverse_digit)`     n
  <lgl>                   <int>
1 FALSE                      18
2 TRUE                      102
```

7.5 Q: How does age look?

For continuous (or mostly continuous) outcomes, such as age (`age_today`), it's helpful to look at all individual reported values, not just mean or median values. We can also use `count()` for this.

```
dat |>
  count(age_today)
```

```
# A tibble: 21 x 2
   age_today     n
       <dbl> <int>
 1        25     6
 2        26     6
 3        27     7
 4        28     6
 5        29     7
 6        30     3
 7        31     6
 8        32     6
 9        33     6
10        34     6
# i 11 more rows
```

If any of these values appear to be typos, implausible, or otherwise incorrect, what do we do next? Generally, a good approach is to

1. Talk to the study coordinator;
2. If required, they can correct the data directly in redcap, correctly documenting the change;
3. Then, the data analyst can re-export the data, and rerun the analysis.

8 Data cleaning and Table 1

```
library(tidyverse)
```

```
set.seed(20261019)
n <- 60
dat <- tibble(study_id = 1:(2 * n),
              first = sample(LETTERS, 2 * n, replace = TRUE),
              last = sample(LETTERS, 2 * n, replace = TRUE),
              inclusion_over17 = 1,
              inclusion2 = 1, inclusion3 = 1, inclusion4 = 1,
              exclusion1 = 0, exclusion2 = 0,
              screened = 1,
              informed_consent = rbinom(2 * n, 1, 0.9),
              endpoints =  rbinom(2 * n, 1, 0.9),
              age_today = floor(runif(2 * n, 25, 45)),
              gender = rbinom(2 * n, 2, 0.250),
              video_games = rbinom(2 * n, 4, 0.2),
              randomization_group = rep(0:1, n)) |>
  mutate(reverse_digit = rnorm(2 * n, 3 + 2 * randomization_group, 1.5),
         reverse_digit = round(reverse_digit),
         cold_pressor = rnorm(2 * n, 60 - 10 * randomization_group, 20),
         cold_pressor = round(cold_pressor),
         video_challenge = rbinom(2 * n, 1, 0.2 + 0.3 * randomization_group)) |>
  mutate(across(age_today:video_challenge, ~ ifelse(informed_consent == 1, .x, NA))) |>
  mutate(across(reverse_digit:video_challenge, ~ ifelse(endpoints == 1, .x, NA))) |>
  select(-endpoints)

sm <- dat |>
  select(-(inclusion_over17:screened)) |>
  rename(id = study_id,
         trt = randomization_group,
         age = age_today) |>
  mutate(gender = factor(gender, 0:2, c("F", "M", "O")),
         trt = factor(trt, 0:1, c("control", "intervention")),
```

```
        video_games = factor(video_games, 0:3,
                             c("<30 minutes per day", "30-60", "60-120", "120+"))) |>
  filter(!is.na(trt), !is.na(reverse_digit))
```

To make a smaller cleaned version of the data for analysis, we might consider doing the following:

1. Remove screening variables. Choosing variables to keep is done with `select()`, e.g. `select(var1, var2)`. If you just want to drop a few variables, try `select(-var1, -var2)`.
2. Rename study_id to id, randomization_group to trt, age_today to age. Renaming variables can be performed by typing using `rename()` , like `rename(newname = oldname)`.
3. Give labels to gender, trt and video_games. The command `mutate()` can be used to make various changes to variables. Specifically, in these cases, we need to also use `factor()`. If, for example, we have a 0/1 variable called `var` where 0 means "no" and 1 means "yes", we could write `mutate(var = factor(var, c(0, 1), c("no", "yes"))`. We can also put multiple changes to multiple variables inside `mutate()`.
4. Filter out participants that are not randomized and do not have the primary endpoint. For this, there are two options `filter()` or `filter_out()`. If you use `filter()`, write out conditions which should be met ("inclusion criteria"). Alternately, if you use `filter_out()`, write out conditions which should *not* be met ("exclusion criteria"). Multiple conditions can be separated by a comma, and will be combined with AND (`&`). If you want OR, you could write `|` instead of a comma. For example, to keep female participants aged 45 or older, we could write `filter(gender == 0, age_today >= 45)`.

All of these functions are in the dplyr package.

```
sm <- dat |>
  rename(id = study_id,
         trt = randomization_group,
         age = age_today) |>
  mutate(gender = factor(gender, 0:2, c("F", "M", "O")),
         trt = factor(trt, 0:1, c("control", "intervention")),
         video_games = factor(video_games, 0:3,
                              c("<30 minutes per day", "30-60", "60-120", "120+"))) |>
  filter(!is.na(trt), !is.na(reverse_digit))
```

Characteristic	control N = 49[1]	intervention N = 53[1]
age	37.0 (29.0, 41.0)	32.0 (28.0, 39.0)
gender		
F	32 (65%)	27 (51%)
M	13 (27%)	18 (34%)
O	4 (8.2%)	8 (15%)
video_games		
<30 minutes per day	19 (39%)	22 (42%)
30-60	19 (39%)	20 (38%)
60-120	9 (18%)	10 (19%)
120+	2 (4.1%)	1 (1.9%)

[1]Median (Q1, Q3); n (%)

8.1 Make a demographics table

Now that we've cleaned the dataset, let's make a table of baseline characteristics. From the dataset we made in the last step, we probably want to include `age, gender, video_games` and report summary statistics by `trt`. Using the R package `gtsummary`, we can do this with the function `tbl_summary()` (see also the introduction on the `gtsummary` website).

```
library(gtsummary)
sm |>
  select(trt, age, gender, video_games) |>
  tbl_summary(by = trt)
```

To clean up or improve the table, try customizing it with options like `label`, `value`, `digit` or `statistic`. Use the `forcats` package gives us more ways to deal with factor variables, in this case, the `fct_recode()` function for change labels on the levels of the factor.

```
library(forcats)
sm |>
  select(trt, age, gender, video_games) |>
  mutate(video_games = fct_recode(video_games, "<30" = "<30 minutes per day")) |>
  tbl_summary(by = trt,
              label = list(video_games = "video games (minutes / day)",
                           gender ~ "gender (F)"),
              value = list(gender ~ "F"),
              digits = list(age ~ 1),
              statistic = list(age ~ "{mean} ± {sd}"))
```

Characteristic	**control** N = 49[1]	**intervention** N = 53[1]
age	35.1 ± 6.3	33.5 ± 6.1
gender (F)	32 (65%)	27 (51%)
video games (minutes / day)		
<30	19 (39%)	22 (42%)
30-60	19 (39%)	20 (38%)
60-120	9 (18%)	10 (19%)
120+	2 (4.1%)	1 (1.9%)

[1]Mean ± SD; n (%)

9 Visualization using `ggplot2`

A very flexible approach to graphing in R uses the `ggplot2` package (take a look at the overview on the package website).

```
library(tidyverse)
theme_set(theme_bw(base_size = 12))
```

> **Note**
>
> Why `gg`? GG stands for "grammar of graphics", as discussed by Wickham (2021). Loading `tidyverse` also gives you `ggplot2`.

Let's work through an example first, using some data we have on weight and flipper length of 3 different species of penguins.

```
# install.packages("palmerpenguins")
library(palmerpenguins)
data(penguins)

penguins |>
  select(species, body_mass_g, flipper_length_mm)
```

```
# A tibble: 344 x 3
   species body_mass_g flipper_length_mm
   <fct>         <int>             <int>
 1 Adelie         3750               181
 2 Adelie         3800               186
 3 Adelie         3250               195
 4 Adelie           NA                NA
 5 Adelie         3450               193
 6 Adelie         3650               190
 7 Adelie         3625               181
 8 Adelie         4675               195
 9 Adelie         3475               193
10 Adelie         4250               190
# i 334 more rows
```

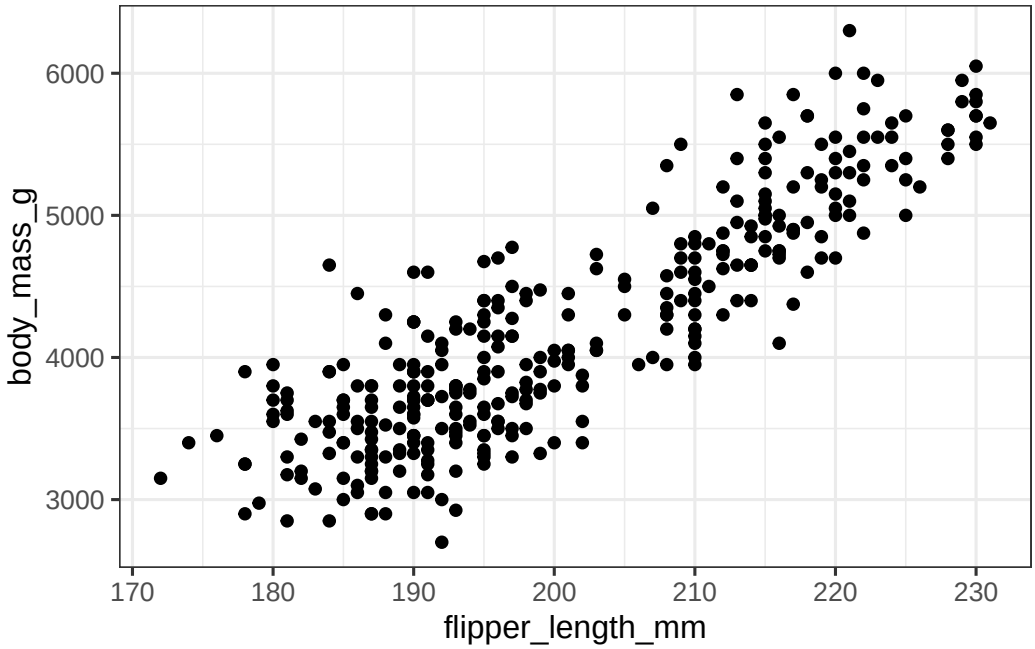
Suppose we want to make a scatter plot with flipper length on the x-axis, and body mass on the y-axis. We set this information up in the `ggplot()` function like this:

```
ggplot(aes(x = ..., y = ...),
       data = ...)
```

and then add (with `+`) `geom_point()` to say we want to show the data with points. Note the warning which points out that data for 2 animals were missing and were not included in the plot.

```
ggplot(aes(x = flipper_length_mm, y = body_mass_g),
       data = penguins) +
  geom_point()
```

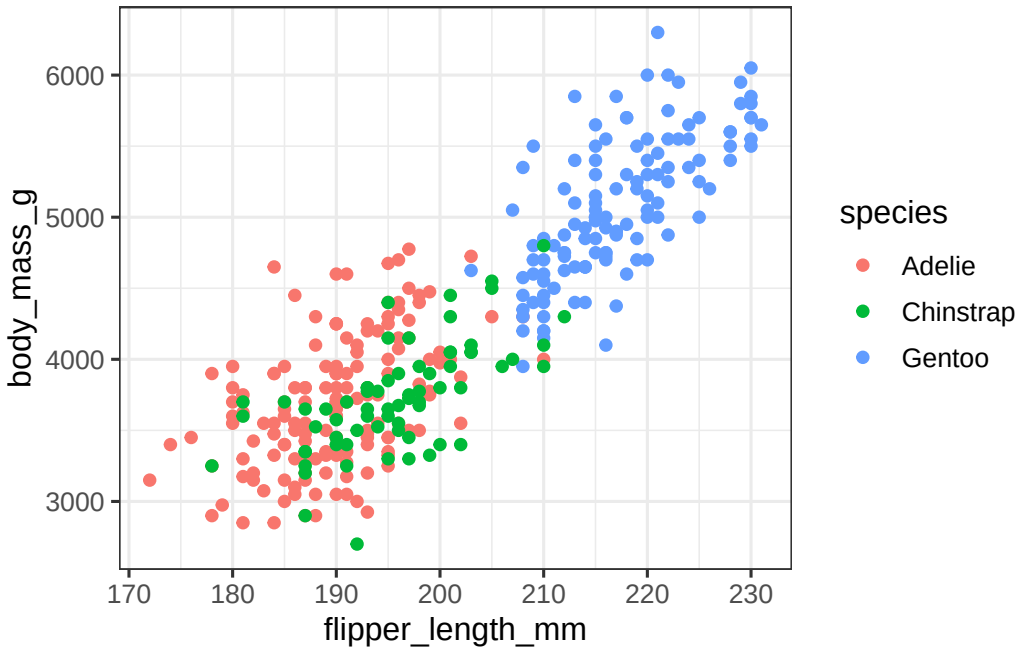
```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



We cannot see which animal is which species yet, but if we wanted to make each species a different color, we can add a color option:

```
ggplot(aes(x = flipper_length_mm, y = body_mass_g,
           color = species),
       data = penguins) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



Adding a few other options, we have a plot that could be used later in a publication or presentation.

```
ggplot(aes(x = flipper_length_mm, y = body_mass_g,
           color = species, shape = species),
       data = penguins) +
  geom_point() +
  xlab("flipper length (mm)") +
  ylab("body mass (g)") +
  guides(color = guide_legend(NULL),
         shape = guide_legend(NULL)) +
  theme(legend.position = "inside",
        legend.position.inside = c(0.15, 0.9))
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```

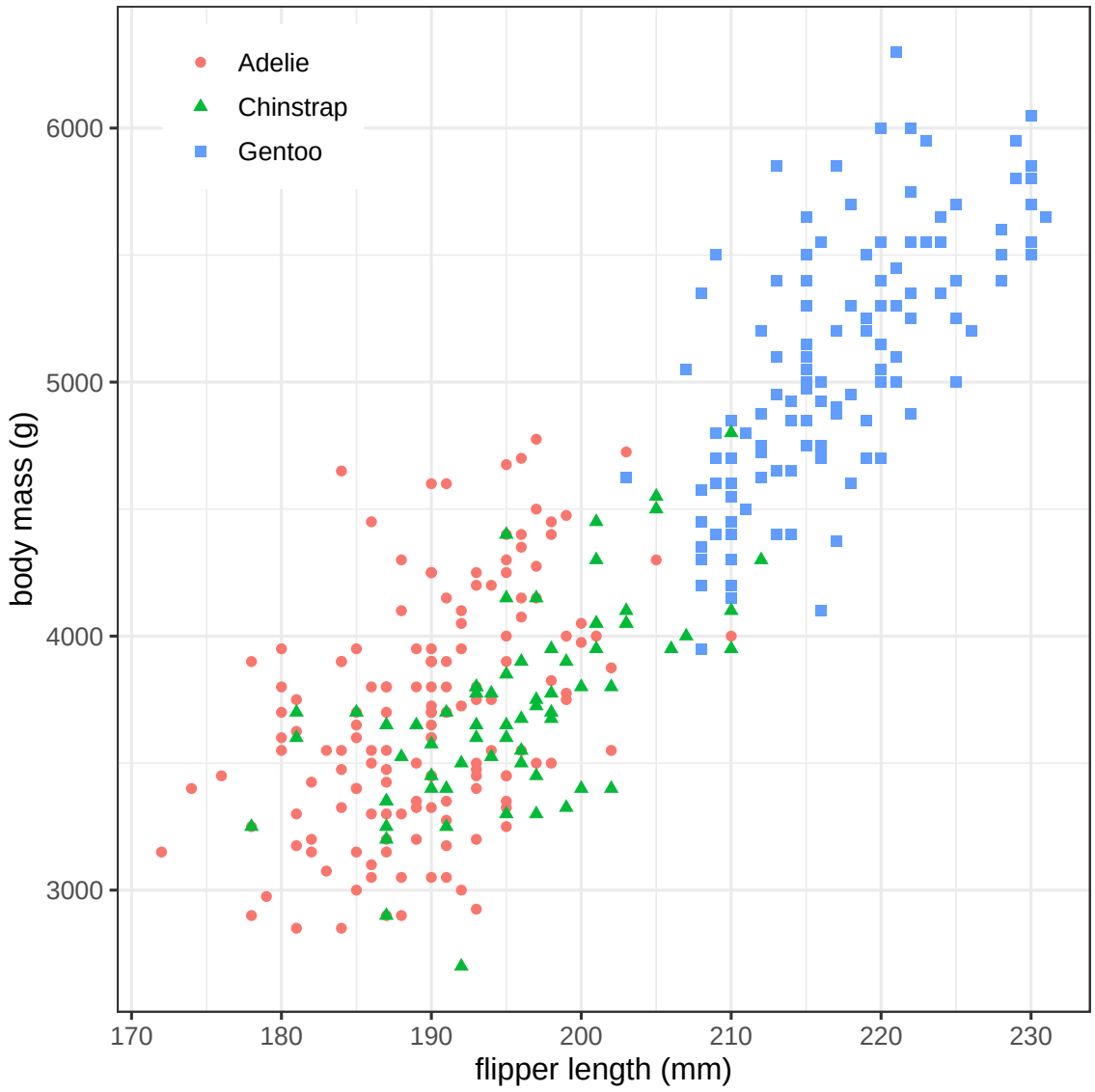
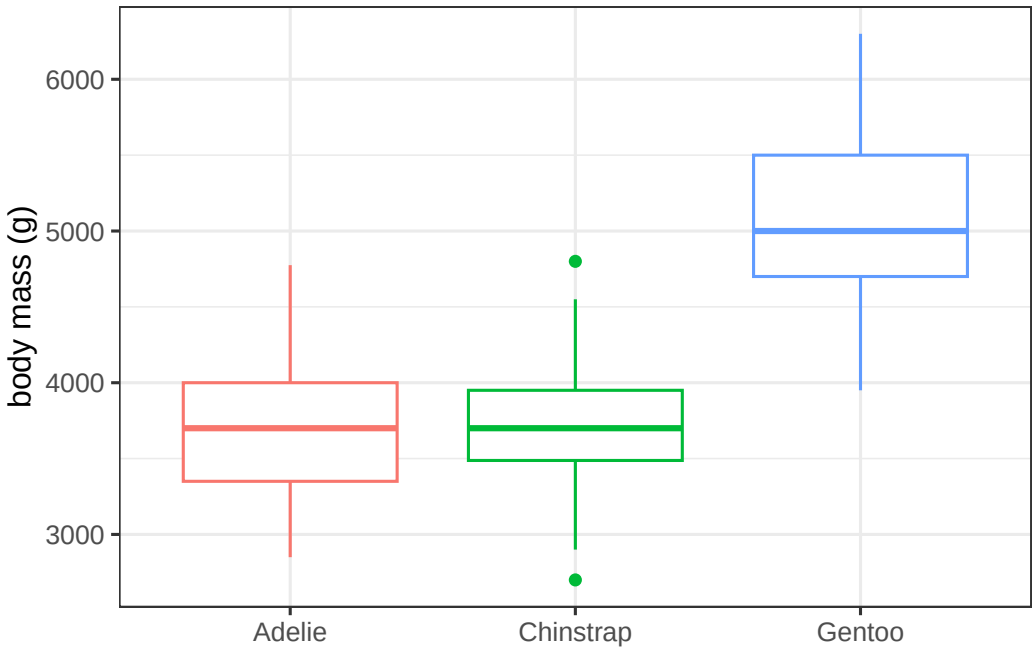


Figure 9.1: publication-ready figure

Let's see how to make a boxplot.

```r
ggplot(aes(x = species, y = body_mass_g,
           color = species),
       data = penguins) +
  geom_boxplot() +
  xlab(NULL) + ylab("body mass (g)") +
  guides(color = "none")
```

```
Warning: Removed 2 rows containing non-finite outside the scale range
(`stat_boxplot()`).
```



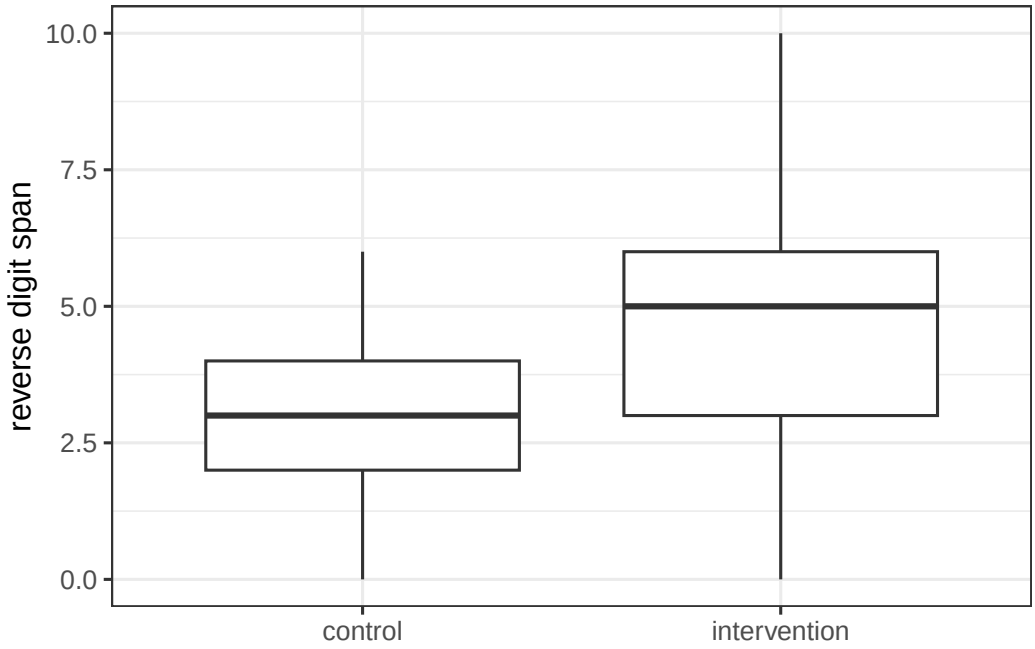
Now let's try to make a boxplot for the primary outcome in our RCT, by treatment.

```r
set.seed(20261019)
n <- 60
dat <- tibble(study_id = 1:(2 * n),
              first = sample(LETTERS, 2 * n, replace = TRUE),
              last = sample(LETTERS, 2 * n, replace = TRUE),
              inclusion_over17 = 1,
              inclusion2 = 1, inclusion3 = 1, inclusion4 = 1,
              exclusion1 = 0, exclusion2 = 0,
              screened = 1,
              informed_consent = rbinom(2 * n, 1, 0.9),
```

```
              endpoints =  rbinom(2 * n, 1, 0.9),
              age_today = floor(runif(2 * n, 25, 45)),
              gender = rbinom(2 * n, 2, 0.250),
              video_games = rbinom(2 * n, 4, 0.2),
              randomization_group = rep(0:1, n)) |>
  mutate(reverse_digit = rnorm(2 * n, 3 + 2 * randomization_group, 1.5),
         reverse_digit = round(reverse_digit),
         cold_pressor = rnorm(2 * n, 60 - 10 * randomization_group, 20),
         cold_pressor = round(cold_pressor),
         video_challenge = rbinom(2 * n, 1, 0.2 + 0.3 * randomization_group)) |>
  mutate(across(age_today:video_challenge, ~ ifelse(informed_consent == 1, .x, NA))) |>
  mutate(across(reverse_digit:video_challenge, ~ ifelse(endpoints == 1, .x, NA))) |>
  select(-endpoints)

sm <- dat |>
  select(-(inclusion_over17:screened)) |>
  rename(id = study_id,
         trt = randomization_group,
         age = age_today) |>
  mutate(gender = factor(gender, 0:2, c("F", "M", "O")),
         trt = factor(trt, 0:1, c("control", "intervention")),
         video_games = factor(video_games, 0:3,
                              c("<30 minutes per day", "30-60", "60-120", "120+"))) |>
  filter(!is.na(trt), !is.na(reverse_digit))
```

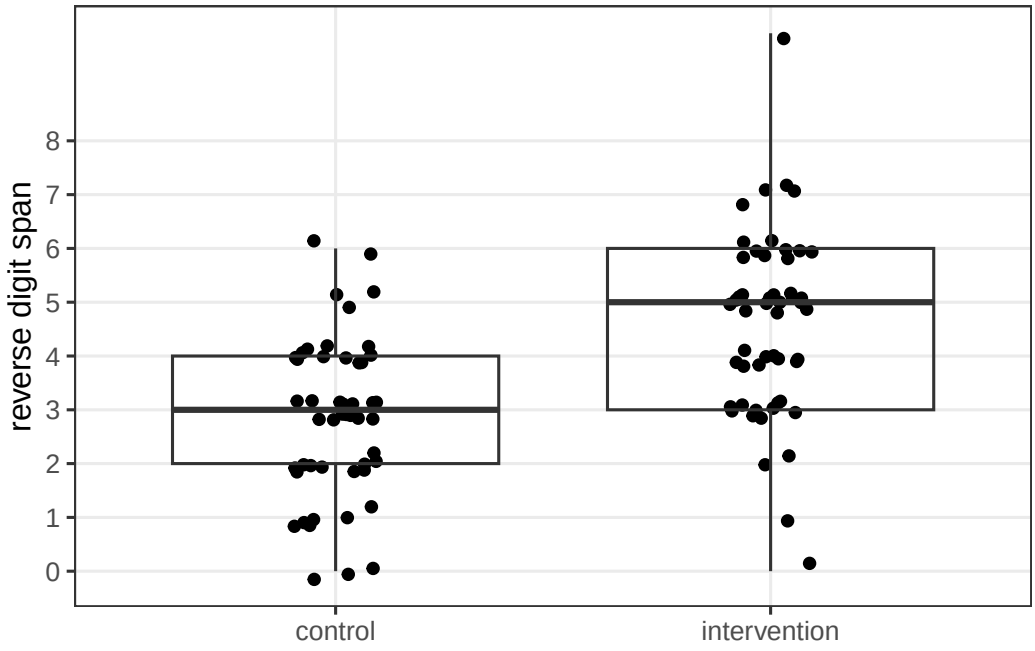
```
sm |>
  ggplot(aes(trt, reverse_digit)) +
  geom_boxplot() +
  xlab(NULL) + ylab("reverse digit span")
```



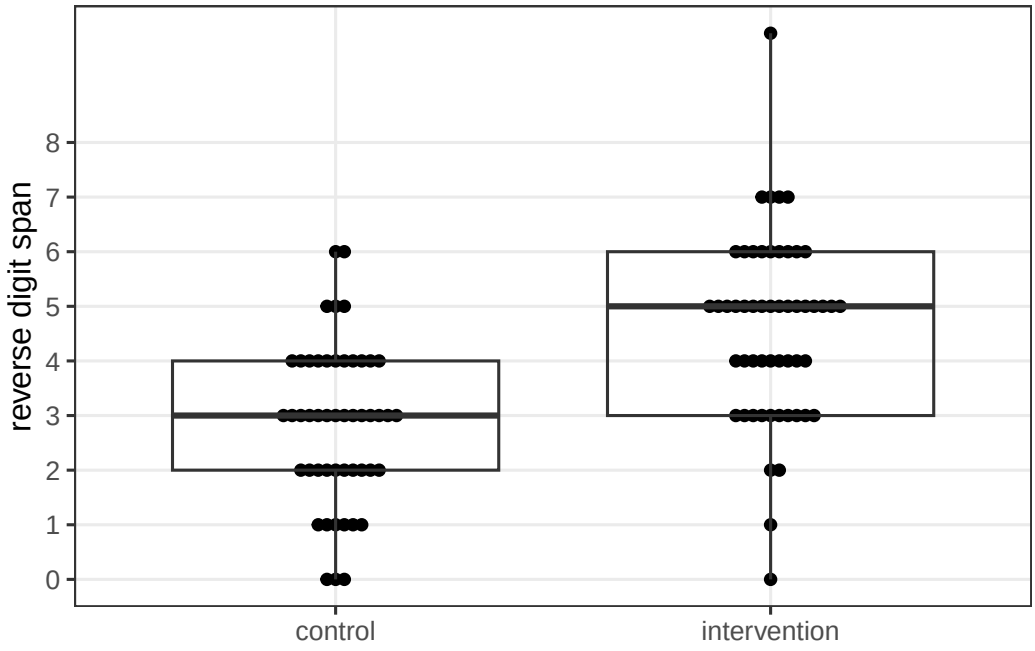
In smaller studies, it's helpful to plot the observations as well as the summary boxplot. One good option for this is with a beeswarm plot, or by using `geom_jitter`. If there's not too much overlap in the outcome values between participants, `geom_point` might also work.

```
# install.packages("ggbeeswarm")
library(ggbeeswarm)

sm |>
  ggplot(aes(trt, reverse_digit)) +
  geom_jitter(width = 0.1, height = 0.2) +
  geom_boxplot(fill = NA, outliers = FALSE) +
  xlab(NULL) + ylab("reverse digit span") +
  scale_y_continuous(breaks = 0:8,
                     minor_breaks = NULL)
```




```
sm |>
  ggplot(aes(trt, reverse_digit)) +
  geom_beeswarm() +
  geom_boxplot(fill = NA, outliers = FALSE) +
  xlab(NULL) + ylab("reverse digit span") +
  scale_y_continuous(breaks = 0:8,
                     minor_breaks = NULL)
```



To save the graph, try the `ggsave()` function. It will save the most recent graph by default.

```
ggsave("primary_beeswarm.jpg")
```

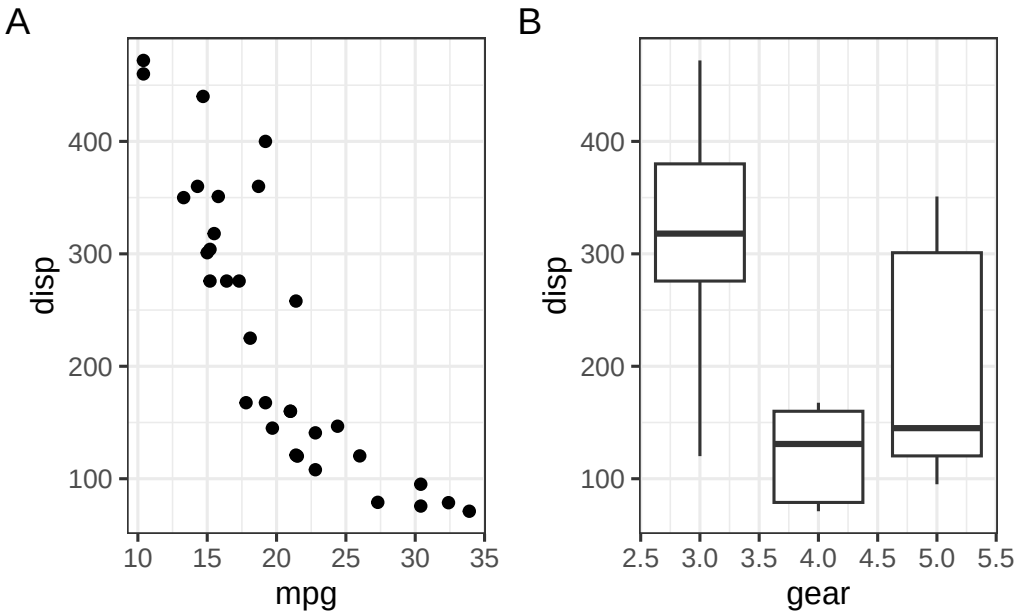
When you need to combine graphs, `patchwork` is great.

```
# install.packages("patchwork")

library(patchwork)

p1 <- ggplot(mtcars) +
  geom_point(aes(mpg, disp))
p2 <- ggplot(mtcars) +
  geom_boxplot(aes(gear, disp, group = gear))

p1 + p2 + plot_annotation(tag_levels = "A")
```


A
B
disp
mpg
gear
400
300
200
100
10
15
20
25
30
35
2.5
3.0
3.5
4.0
4.5
5.0
5.5

10 Comparing and testing outcomes

From the boxplot, it looked like the intervention arm had more remembered digits than the control arm. Now we summarize these values, and test for differences between the two groups.

```
library(tidyverse)
theme_set(theme_bw(base_size = 12))
library(gtsummary)
```

```
set.seed(20261019)
n <- 60
dat <- tibble(study_id = 1:(2 * n),
              first = sample(LETTERS, 2 * n, replace = TRUE),
              last = sample(LETTERS, 2 * n, replace = TRUE),
              inclusion_over17 = 1,
              inclusion2 = 1, inclusion3 = 1, inclusion4 = 1,
              exclusion1 = 0, exclusion2 = 0,
              screened = 1,
              informed_consent = rbinom(2 * n, 1, 0.9),
              endpoints =  rbinom(2 * n, 1, 0.9),
              age_today = floor(runif(2 * n, 25, 45)),
              gender = rbinom(2 * n, 2, 0.250),
              video_games = rbinom(2 * n, 4, 0.2),
              randomization_group = rep(0:1, n)) |>
  mutate(reverse_digit = rnorm(2 * n, 3 + 2 * randomization_group, 1.5),
         reverse_digit = round(reverse_digit),
         cold_pressor = rnorm(2 * n, 60 - 10 * randomization_group, 20),
         cold_pressor = round(cold_pressor),
         video_challenge = rbinom(2 * n, 1, 0.2 + 0.3 * randomization_group)) |>
  mutate(across(age_today:video_challenge, ~ ifelse(informed_consent == 1, .x, NA))) |>
  mutate(across(reverse_digit:video_challenge, ~ ifelse(endpoints == 1, .x, NA))) |>
  select(-endpoints)

sm <- dat |>
  select(-(inclusion_over17:screened)) |>
  rename(id = study_id,
```

Characteristic	**control** N = 49[1]	**intervention** N = 53[1]
reverse digit span test	2.84 (1.45)	4.58 (1.69)
cold pressor test	60 (18)	47 (23)
video challenge	5 (10%)	31 (58%)

[1]Mean (SD); n (%)

```
        trt = randomization_group,
        age = age_today) |>
  mutate(gender = factor(gender, 0:2, c("F", "M", "O")),
        trt = factor(trt, 0:1, c("control", "intervention")),
        video_games = factor(video_games, 0:3,
                             c("<30 minutes per day", "30-60", "60-120", "120+"))) |>
  filter(!is.na(trt), !is.na(reverse_digit))
```

Just like we did earlier with the demographics table, let's make a summary table using `tbl_summary()`.

```
sm |>
  select(trt, reverse_digit, cold_pressor, video_challenge) |>
  tbl_summary(by = trt,
              type = list(reverse_digit ~ "continuous"),
              statistic = list(reverse_digit ~ "{mean} ({sd})",
                               cold_pressor ~ "{mean} ({sd})"),
              label = list(reverse_digit ~ "reverse digit span test",
                           cold_pressor ~ "cold pressor test",
                           video_challenge ~ "video challenge"))
```

The difference between these two means is exactly what we should get from a two-sample t-test, or linear regression. If we use linear regression, it's quite easy to get publication-ready output using `tbl_regression()`from the `gtsummary` package.

```
mod <- lm(reverse_digit ~ trt, data = sm)
tbl_regression(mod, conf.int = TRUE)
```

Now try to run a similar model for `cold_pressor`.

```
mod <- lm(cold_pressor ~ trt, data = sm)
tbl_regression(mod, conf.int = TRUE)
```

Characteristic	Beta	95% CI	p-value
trt			
control	—	—	
intervention	1.7	1.1, 2.4	<0.001

Abbreviation: CI = Confidence Interval

Characteristic	Beta	95% CI	p-value
trt			
control	—	—	
intervention	-13	-21, -4.8	0.002

Abbreviation: CI = Confidence Interval

And for binary outcomes like the `video_challenge`, we can run logistic regression. While there is no function in R to run a logistic regression model directly, we can use `glm()` with the option `family = binomial`. To get odds ratios, we put the option `exponentiate = TRUE` in `tbl_regression()`.

```
mod <- glm(video_challenge ~ trt, data = sm,
           family = binomial)
tbl_regression(mod, conf.int = TRUE,
               exponentiate = TRUE)
```

Characteristic	OR	95% CI	p-value
trt			
control	—	—	
intervention	12.4	4.55, 40.4	<0.001

Abbreviations: CI = Confidence Interval, OR = Odds Ratio

11 Back to the beginning: importing data

Most of the time, datasets will not come already loaded in R, but it can be complicated to get the data in correctly.

For `csv` files exported from redcap, `read_csv()` from the `readr` package is a good place to start.

> Tip
>
> The core packages in the `tidyverse` are: `dplyr` (data cleaning), `ggplot2` (visualization), `readr` (importing data), `forcats` (factors), `stringr` (strings), `lubridate` (times and dates), `tibble` (alternative to `data.frame`), `tidyr` (pivoting between long and wide formats), and `purrr` (dealing with functions and vectors).

```
library(tidyverse)
```

```
dat <- read_csv("somedata.csv")
```

If you export the dataset with decimal points, `.`, you'll find that `read_csv()` should work just fine. If you exported it with decimal commas, `,`, or if you've saved data in Excel on your computer using, say, a Swiss locale, try `read_csv2()`. Find out more about `read_csv()` in R4DS, and more about locales in the `readr` documentation.

> Tip
>
> Why not `read.csv()`?

If you are not exporting data from redcap or another electronic data capture system, consider organizing your data to enable efficient data analysis Broman and Woo (2018).

If you're working with an Excel spreadsheet, `.xlsx`, the `readxl` package will help.

For formats from other statistical programs (for example from SPSS, SAS, or STATA), try the `haven` package.

12 Summary

In summary, this book has no content whatsoever.

```
1 + 1
```

```
[1] 2
```

References

Bland, JM, and DG Altman. 2000. "Statistics Notes. The Odds Ratio." *BMJ* 320 (7247). https://doi.org/10.1136/bmj.320.7247.1468.

Broman, KW, and KH Woo. 2018. "Data Organization in Spreadsheets." *The American Statistician* 72 (1): 2–10. https://doi.org/10.1080/00031305.2017.1375989.

Greenland, S, SJ Senn, KJ Rothman, et al. 2016. "Statistical Tests, p Values, Confidence Intervals, and Power: A Guide to Misinterpretations." *Eur J Epidemiol* 31 (4): 337–50. https://doi.org/10.1007/s10654-016-0149-3.

Grolemund, G. 2014. *Hands-on Programming with r*. O'Reilly. https://rstudio-education.github.io/hopr/.

Held, L, K Rufibach, and B Seifert. 2013. *Medizinische Statistik : Konzepte, Methoden, Anwendungen*. Pearson.

Kirkwood, B, and JAC Sterne. 2003. *Essential Medical Statistics*. 2nd ed. Blackwell.

Muff, S, EB Nilsen, RB O'Hara, and CR Nater. 2022. "Rewriting Results Sections in the Language of Evidence." *Trends Ecol Evol* 37 (3): 203–10. https://doi.org/10.1016/j.tree.2021.10.009.

Puhan, MA, C Suarez, C Lo Cascio, A Zahn, M Heitz, and O Braendli. 2006. "Didgeridoo Playing as Alternative Treatment for Obstructive Sleep Apnoea Syndrome: Randomised Controlled Trial." *BMJ* 332 (7536): 7536. https://doi.org/10.1136/bmj.38705.470590.55.

Radtke, T, V von Wyl, SR Haile, S Rohrmann, A Frei, and MA Puhan. 2024. "Evidence-Based Coaching of Core Competencies in Epidemiology, Using the Framework of Randomized Controlled Trials: The Zurich Approach." *Int J Epidemiol.* 53 (3): dyae075. https://doi.org/10.1093/ije/dyae075.

Rovettaa, A, MA Mansournia, SD Stovitz, WM Adams, and S Greenland. 2025. "Interpreting p Values and Interval Estimates Based on Practical Relevance: Guidance for the Sports Medicine Clinician." *Br J Sports* 59 (24): bjsports-2024-109357. https://doi.org/10.1136/bjsports-2024-109357.

Senn, S. 2002. *Cross-over Trials in Clinical Research*. Wiley.

Senn, S. 2021. *Statistical Issues in Drug Development*. 3rd ed. Wiley.

Wickham, H. 2021. *Ggplot2: Elegant Graphics for Data Analysis*. 3e ed. Springer. https://r4ds.hadley.nz.

Wickham, H, M Cetinkaya-Rundel, and G Grolemund. 2023. *R for Data Science*. 2e ed. O'Reilly. https://r4ds.hadley.nz.

Zimmerman, DW. 2004. "A Note on Preliminary Tests of Equality of Variances." *British Journal of Mathematical and Statistical Psychology* 57: 173–81. https://doi.org/https://doi.org/10.1348/000711004849222.